

Jamboree Education

September 30, 2025

0.1 About Jamboree

Jamboree has helped thousands of students make it to top colleges abroad. Be it GMAT, GRE or SAT, their unique problem-solving methods ensure maximum scores with minimum effort.

They recently launched a feature where students/learners can come to their website and check their probability of getting into the IVY league college. This feature estimates the chances of graduate admission from an Indian perspective.

0.2 Business Aim of the Case study :

1. Understanding what factors are important in graduate admissions and how these factors are interrelated among themselves.
2. It will also help predict one's chances of admission given the rest of the variables.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

0.3 Exploratory Data Analysis

0.4 Observations on shape of data, data types of all the attributes, statistical summary

```
[2]: df = pd.read_csv("Jamboree_Admission.csv")
df.head()
```

```
[2]:
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	\
0	1	337	118	4	4.5	4.5	9.65	
1	2	324	107	4	4.0	4.5	8.87	
2	3	316	104	3	3.0	3.5	8.00	
3	4	322	110	3	3.5	2.5	8.67	
4	5	314	103	2	2.0	3.0	8.21	

	Research	Chance of Admit
0	1	0.92
1	1	0.76
2	1	0.72

3	1	0.80
4	0	0.65

```
[5]: df.shape
```

```
[5]: (500, 9)
```

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Serial No.            500 non-null   int64
1   GRE Score              500 non-null   int64
2   TOEFL Score            500 non-null   int64
3   University Rating      500 non-null   int64
4   SOP                    500 non-null   float64
5   LOR                    500 non-null   float64
6   CGPA                   500 non-null   float64
7   Research               500 non-null   int64
8   Chance of Admit        500 non-null   float64
dtypes: float64(4), int64(5)
memory usage: 35.3 KB
```

```
[7]: df.isnull().sum()/len(df)*100
```

```
[7]: Serial No.            0.0
GRE Score                0.0
TOEFL Score              0.0
University Rating        0.0
SOP                      0.0
LOR                      0.0
CGPA                     0.0
Research                 0.0
Chance of Admit          0.0
dtype: float64
```

Given data set have only int and float data types and didnt have any Null values.

```
[8]: df.describe()
```

```
[8]:
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP \
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	250.500000	316.472000	107.192000	3.114000	3.374000
std	144.481833	11.295148	6.081868	1.143512	0.991004
min	1.000000	290.000000	92.000000	1.000000	1.000000

25%	125.750000	308.000000	103.000000	2.000000	2.500000
50%	250.500000	317.000000	107.000000	3.000000	3.500000
75%	375.250000	325.000000	112.000000	4.000000	4.000000
max	500.000000	340.000000	120.000000	5.000000	5.000000

	LOR	CGPA	Research	Chance of Admit
count	500.00000	500.00000	500.00000	500.00000
mean	3.48400	8.57644	0.56000	0.72174
std	0.92545	0.60481	0.49688	0.14114
min	1.00000	6.80000	0.00000	0.34000
25%	3.00000	8.12750	0.00000	0.63000
50%	3.50000	8.56000	1.00000	0.72000
75%	4.00000	9.04000	1.00000	0.82000
max	5.00000	9.92000	1.00000	0.97000

```
[9]: df.columns
```

```
[9]: Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',
        'LOR ', 'CGPA', 'Research', 'Chance of Admit '],
        dtype='object')
```

```
[10]: df["University Rating"].value_counts(normalize=True)*100
```

```
[10]: University Rating
3    32.4
2    25.2
4    21.0
5    14.6
1     6.8
Name: proportion, dtype: float64
```

```
[11]: df["Research"].value_counts(normalize=True)*100
```

```
[11]: Research
1    56.0
0    44.0
Name: proportion, dtype: float64
```

```
[12]: df["SOP"].value_counts(normalize=True)*100
```

```
[12]: SOP
4.0    17.8
3.5    17.6
3.0    16.0
2.5    12.8
4.5    12.6
2.0     8.6
5.0     8.4
```

```

1.5    5.0
1.0    1.2
Name: proportion, dtype: float64

```

```
[13]: df["LOR "].value_counts(normalize=True)*100
```

```

[13]: LOR
3.0    19.8
4.0    18.8
3.5    17.2
4.5    12.6
2.5    10.0
5.0    10.0
2.0     9.2
1.5     2.2
1.0     0.2
Name: proportion, dtype: float64

```

```
[14]: df["Chance of Admit "].value_counts(normalize=True)
```

```

[14]: Chance of Admit
0.71    0.046
0.64    0.038
0.73    0.036
0.72    0.032
0.79    0.032
...
0.60    0.004
0.51    0.004
0.43    0.002
0.39    0.002
0.37    0.002
Name: proportion, Length: 61, dtype: float64

```

Dropping Serial no. column as we don't want our model to develop any understanding using it's values.

```
[15]: df.drop(columns = ["Serial No."], inplace= True)
```

```
[16]: df.describe()
```

```

[16]:      GRE Score  TOEFL Score  University Rating      SOP      LOR  \
count  500.000000  500.000000      500.000000  500.000000  500.000000
mean    316.472000   107.192000        3.114000    3.374000    3.484000
std      11.295148     6.081868        1.143512    0.991004    0.925450
min     290.000000     92.000000        1.000000    1.000000    1.000000
25%     308.000000   103.000000        2.000000    2.500000    3.000000
50%     317.000000   107.000000        3.000000    3.500000    3.500000

```

75%	325.000000	112.000000	4.000000	4.000000	4.000000
max	340.000000	120.000000	5.000000	5.000000	5.000000

	CGPA	Research	Chance of Admit
count	500.000000	500.000000	500.000000
mean	8.576440	0.560000	0.72174
std	0.604813	0.496884	0.14114
min	6.800000	0.000000	0.34000
25%	8.127500	0.000000	0.63000
50%	8.560000	1.000000	0.72000
75%	9.040000	1.000000	0.82000
max	9.920000	1.000000	0.97000

```
[17]: df["University Rating"].value_counts()
```

```
[17]: University Rating
3      162
2      126
4      105
5       73
1       34
Name: count, dtype: int64
```

0.5 Univariate Analysis (distribution plots of all the continuous variable(s) barplots/countplots of all the categorical variables)

```
[18]: fig = sns.displot(df['GRE Score'], kde=False)
plt.title("Distribution of GRE Scores")
plt.show()

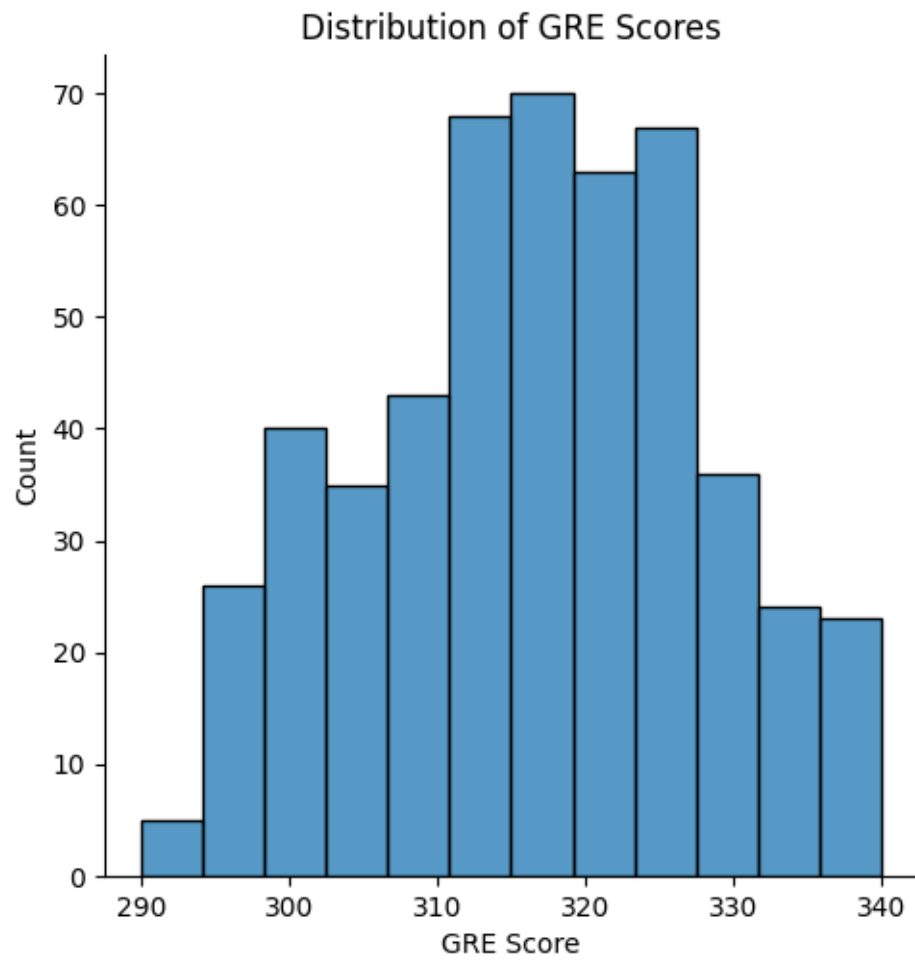
fig = sns.displot(df['TOEFL Score'], kde=False)
plt.title("Distribution of TOEFL Scores")
plt.show()

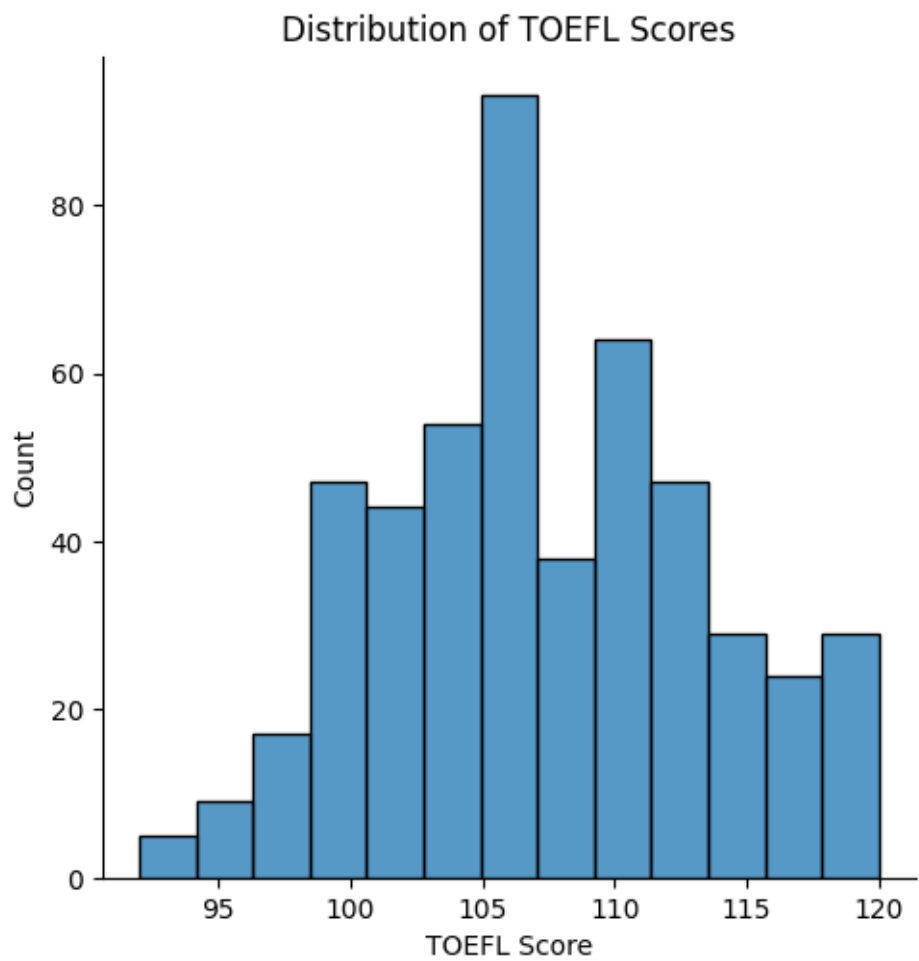
fig = sns.displot(df['University Rating'], kde=False)
plt.title("Distribution of University Rating")
plt.show()

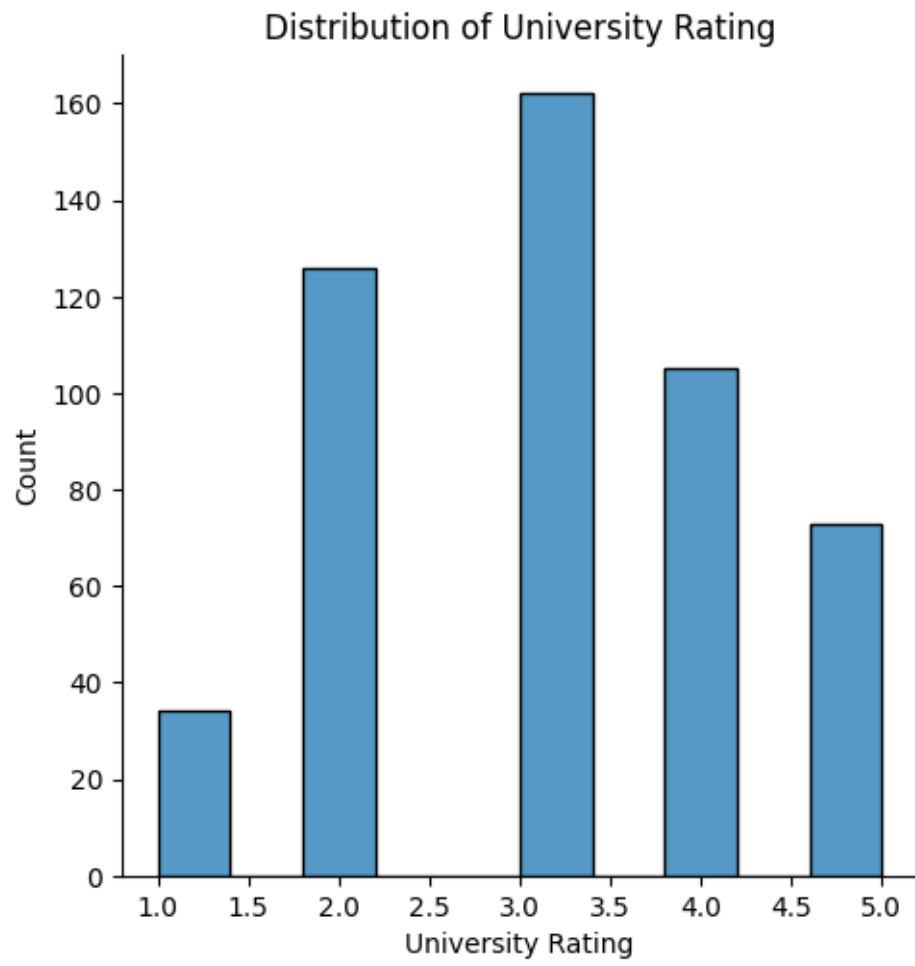
fig = sns.displot(df['SOP'], kde=False)
plt.title("Distribution of SOP Ratings")
plt.show()

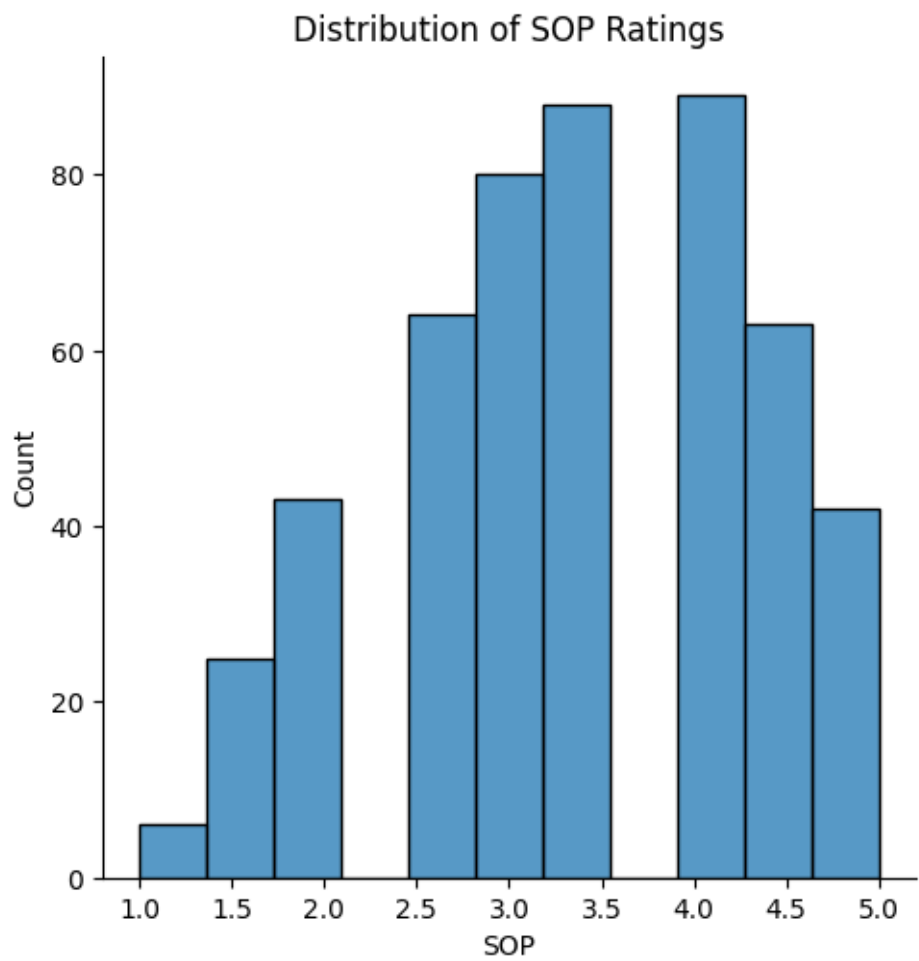
fig = sns.displot(df['CGPA'], kde=False)
plt.title("Distribution of CGPA")
plt.show()

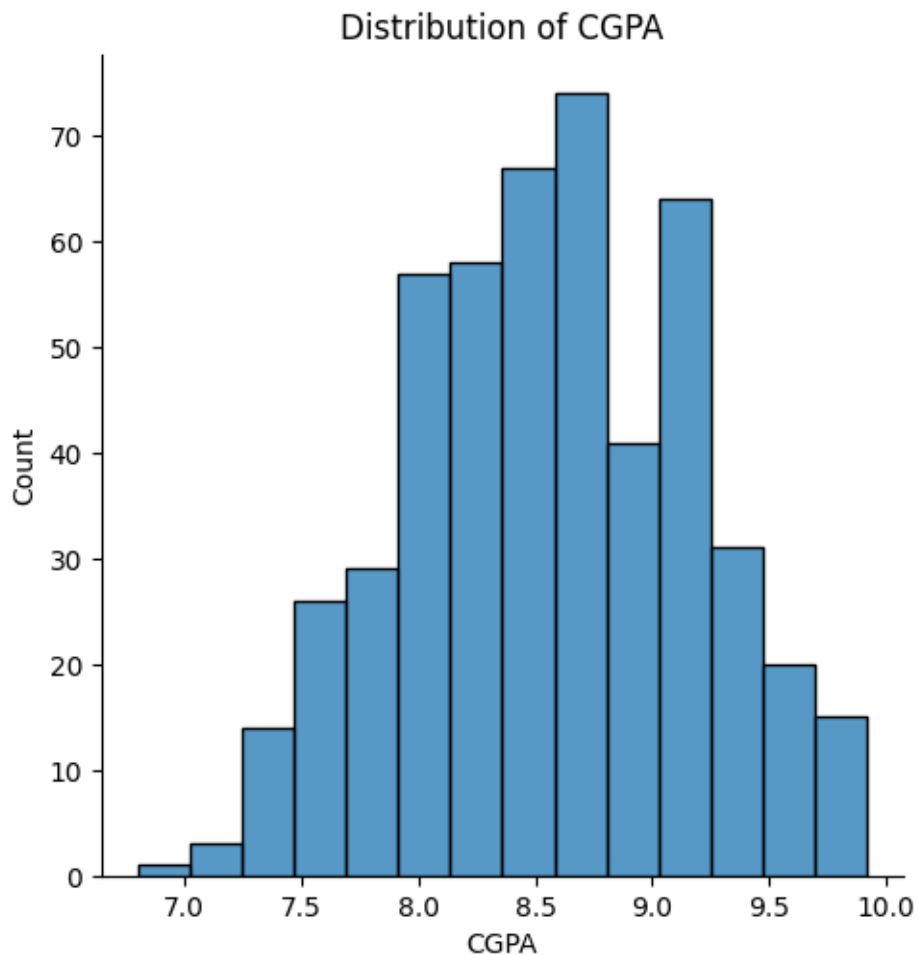
plt.show()
```







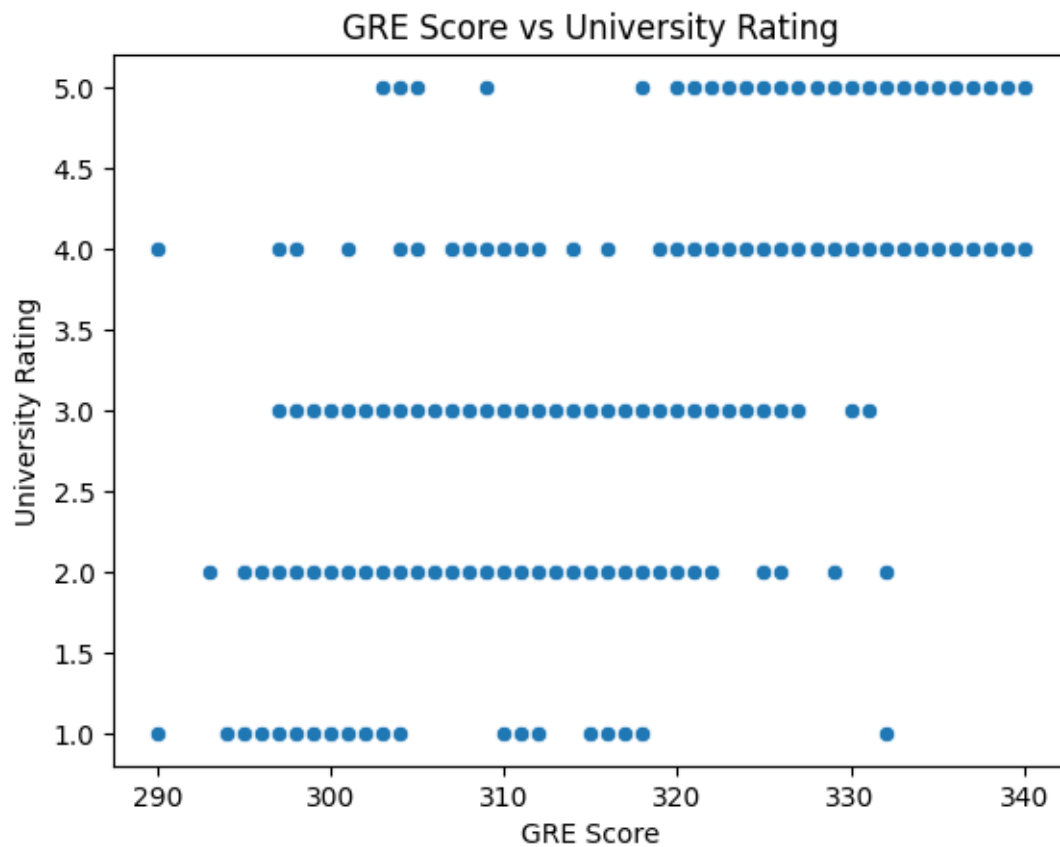




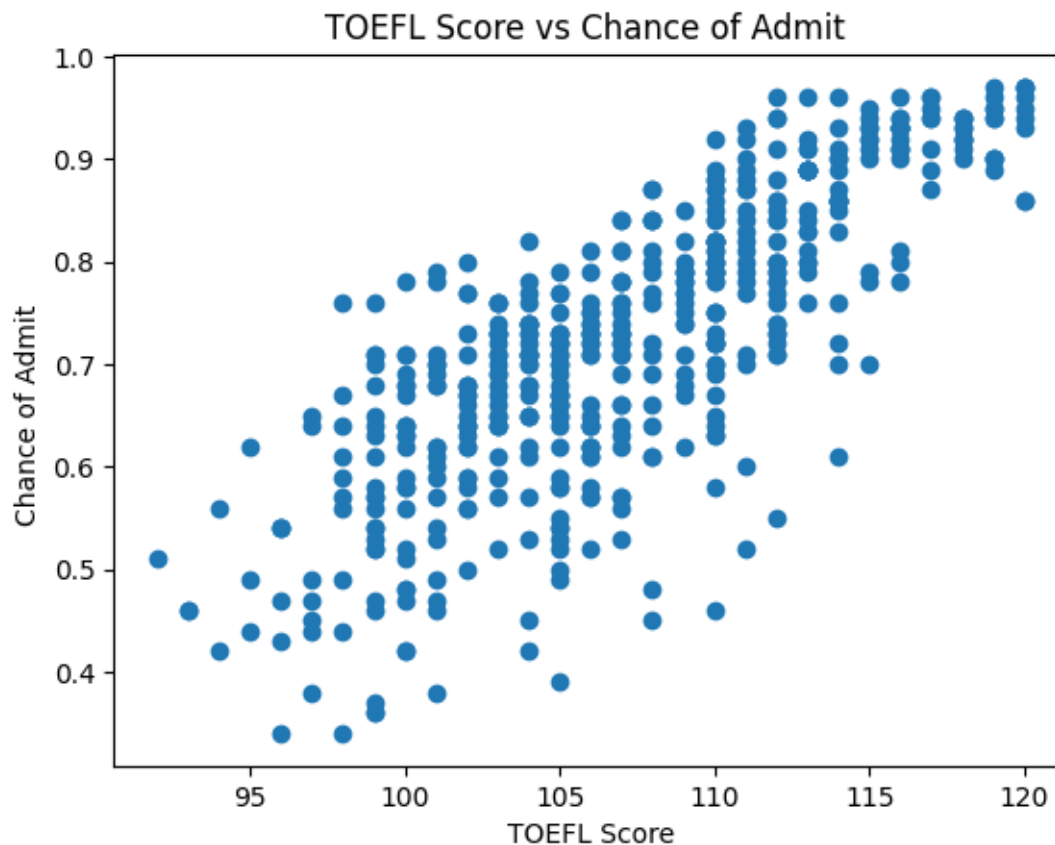
It is clear from the distributions, students with varied merit apply for the university.

Understanding the relation between different factors responsible for graduate admissions

```
[19]: sns.scatterplot(x="GRE Score", y="University Rating", data=df)
plt.title("GRE Score vs University Rating")
plt.show()
```

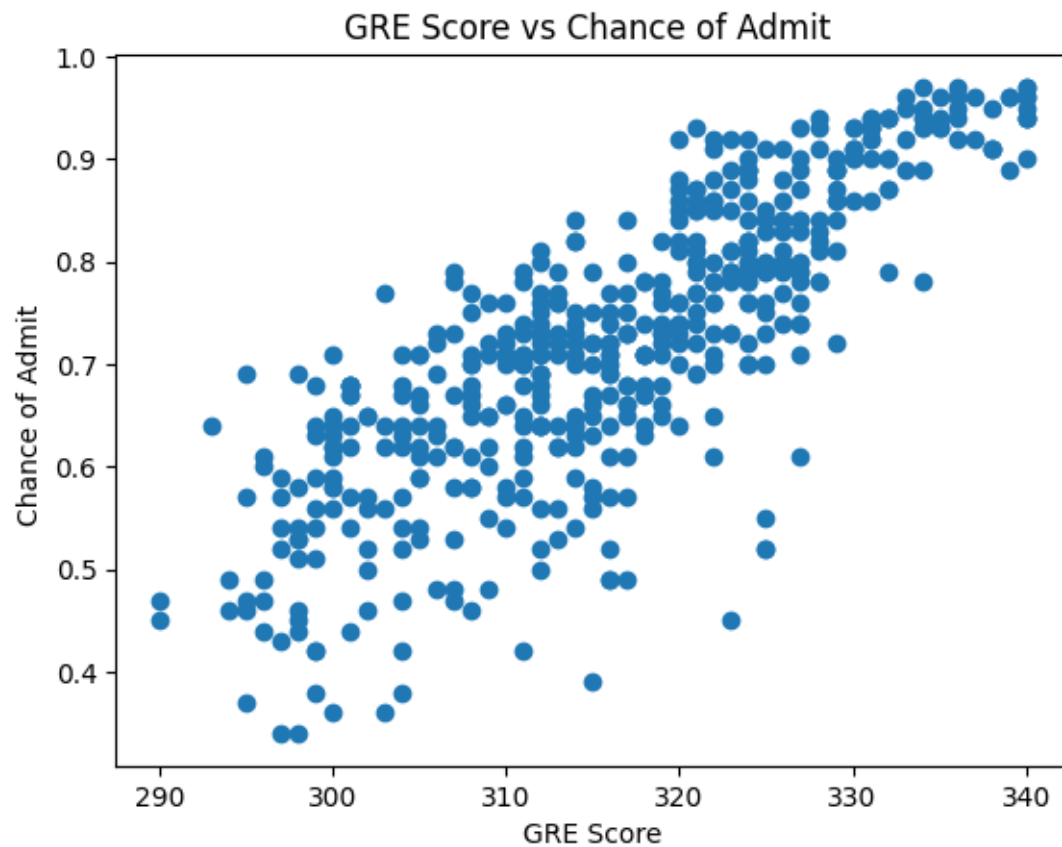


```
[20]: plt.scatter(x = df["TOEFL Score"], y = df["Chance of Admit "])
plt.xlabel("TOEFL Score")
plt.ylabel("Chance of Admit ")
plt.title("TOEFL Score vs Chance of Admit ")
plt.show()
```



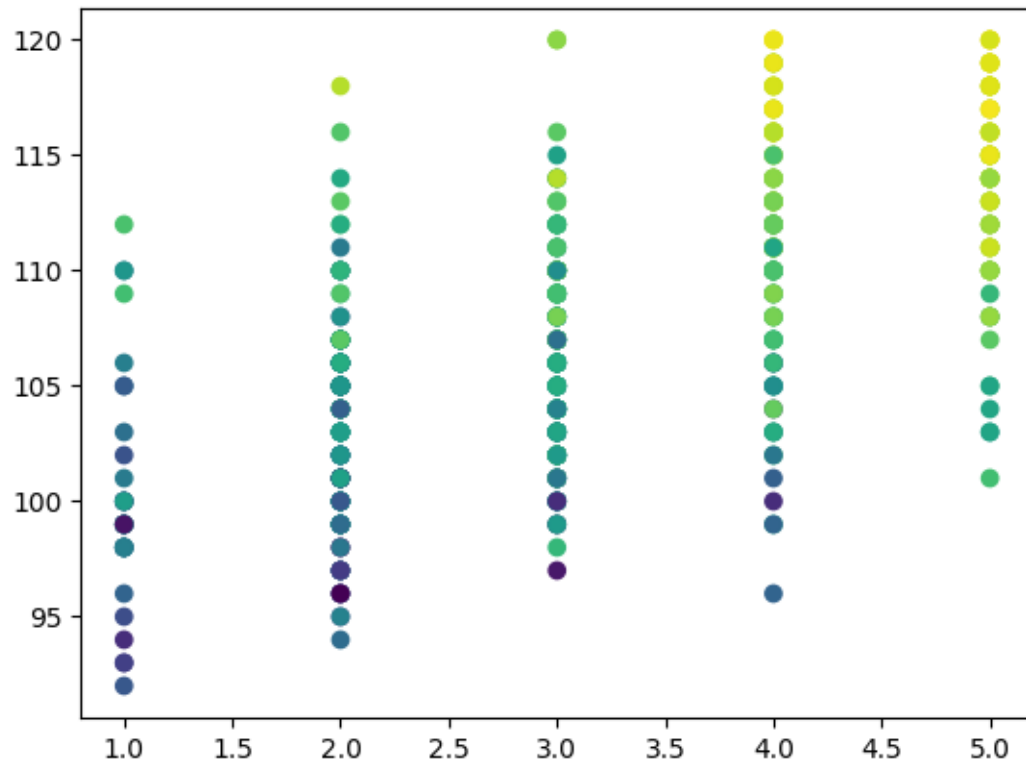
The above graph clearly demonstrates that Higher Ranking Universities are having students with High GRE scores. The chances of admission also can be clearly seen here.

```
[21]: plt.scatter(x = df["GRE Score"], y = df["Chance of Admit "])
plt.xlabel("GRE Score")
plt.ylabel("Chance of Admit ")
plt.title("GRE Score vs Chance of Admit ")
plt.show()
```



```
[22]: plt.scatter(x = df["University Rating"], y = df["TOEFL Score"], c = df["Chance_
↳of Admit "])
```

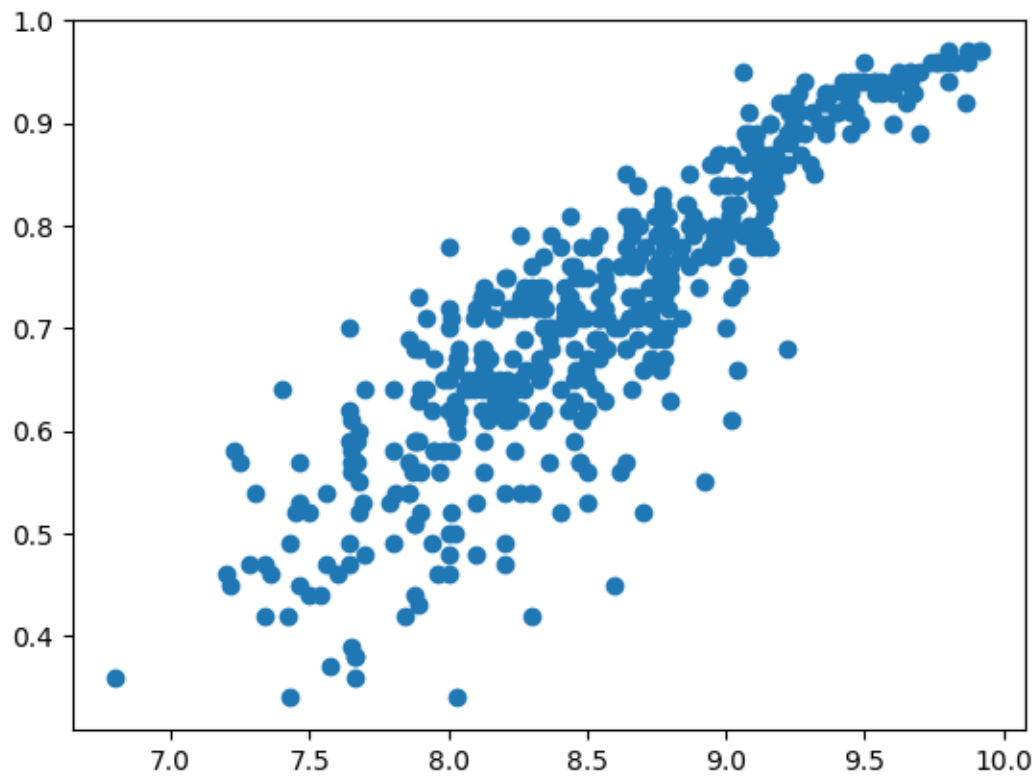
```
[22]: <matplotlib.collections.PathCollection at 0x2b90e33ec10>
```



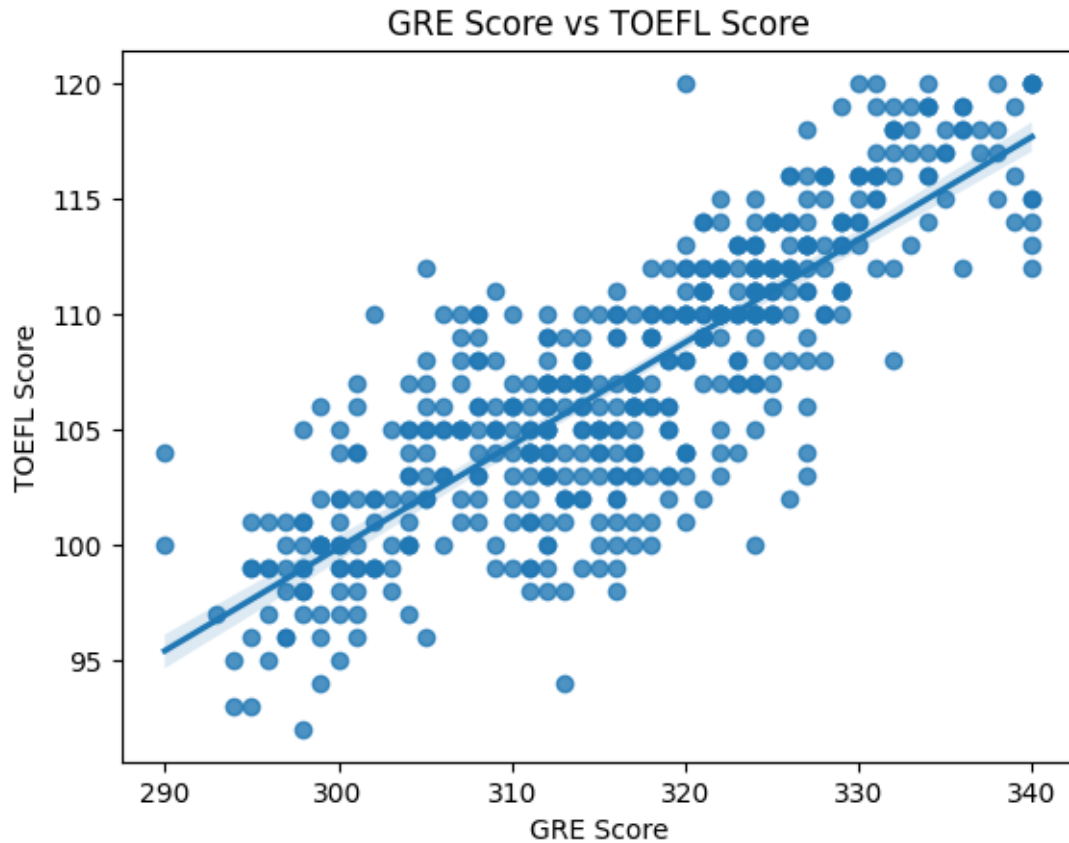
- The Higher Ranking Universities preferred students with High TOEFL scores. The chances of admission also can be clearly seen here.
- The insight looks similar to GRE scores.

```
[24]: plt.scatter(x = df["CGPA"], y = df["Chance of Admit "])
```

```
[24]: <matplotlib.collections.PathCollection at 0x2b90e316d50>
```

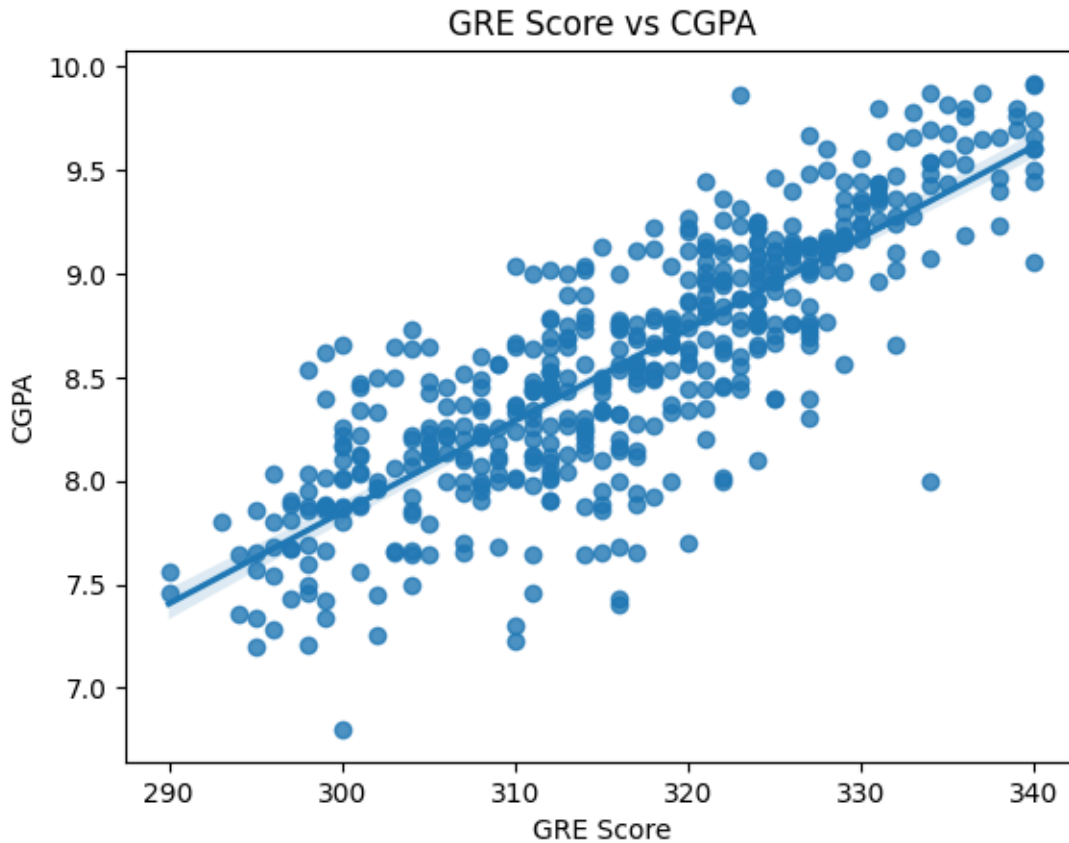


```
[25]: fig = sns.regplot(x="GRE Score", y="TOEFL Score", data=df)
plt.title("GRE Score vs TOEFL Score")
plt.show()
```



People with higher GRE Scores also have higher TOEFL Scores which is justified because both TOEFL and GRE have a verbal section which although not similar are relatable.

```
[26]: fig = sns.regplot(x="GRE Score", y="CGPA", data=df)
plt.title("GRE Score vs CGPA")
plt.show()
```

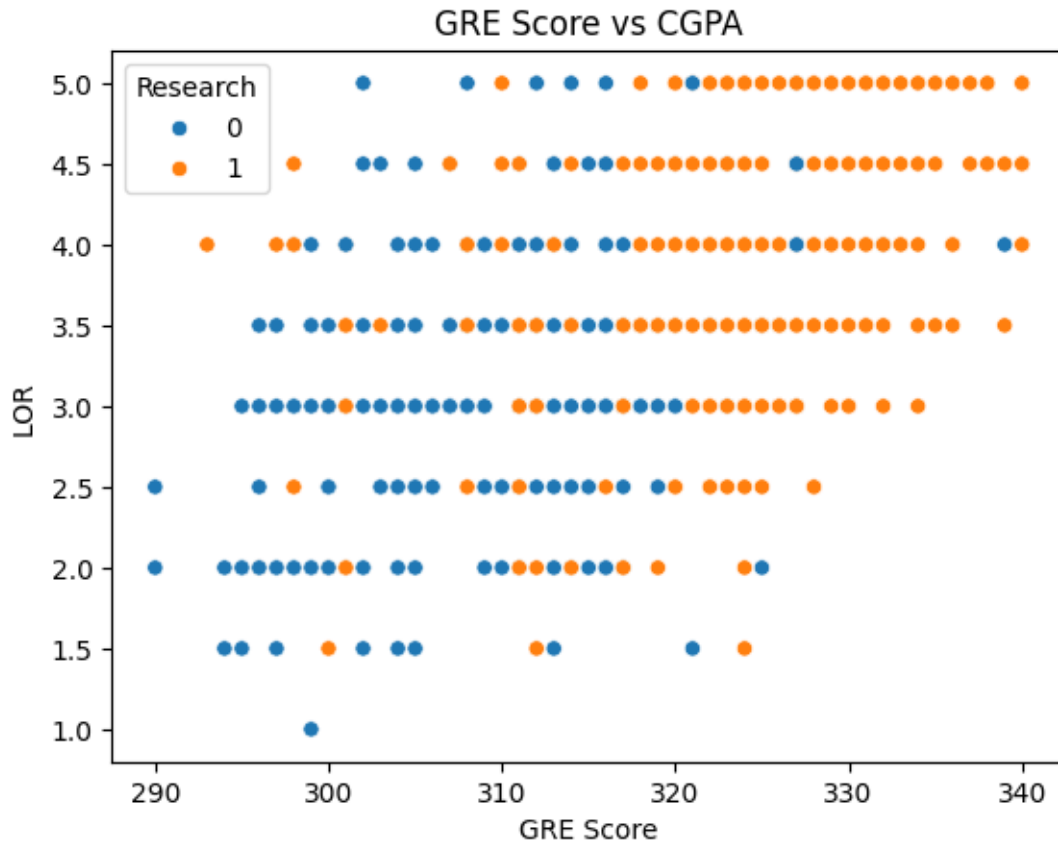
Although there are exceptions, people with higher CGPA usually have higher GRE scores maybe because they are smart or hard working

```
[27]: fig = sns.scatterplot(x="CGPA", y="LOR ", data=df, hue="Research")  
plt.title("GRE Score vs CGPA")  
plt.show()
```



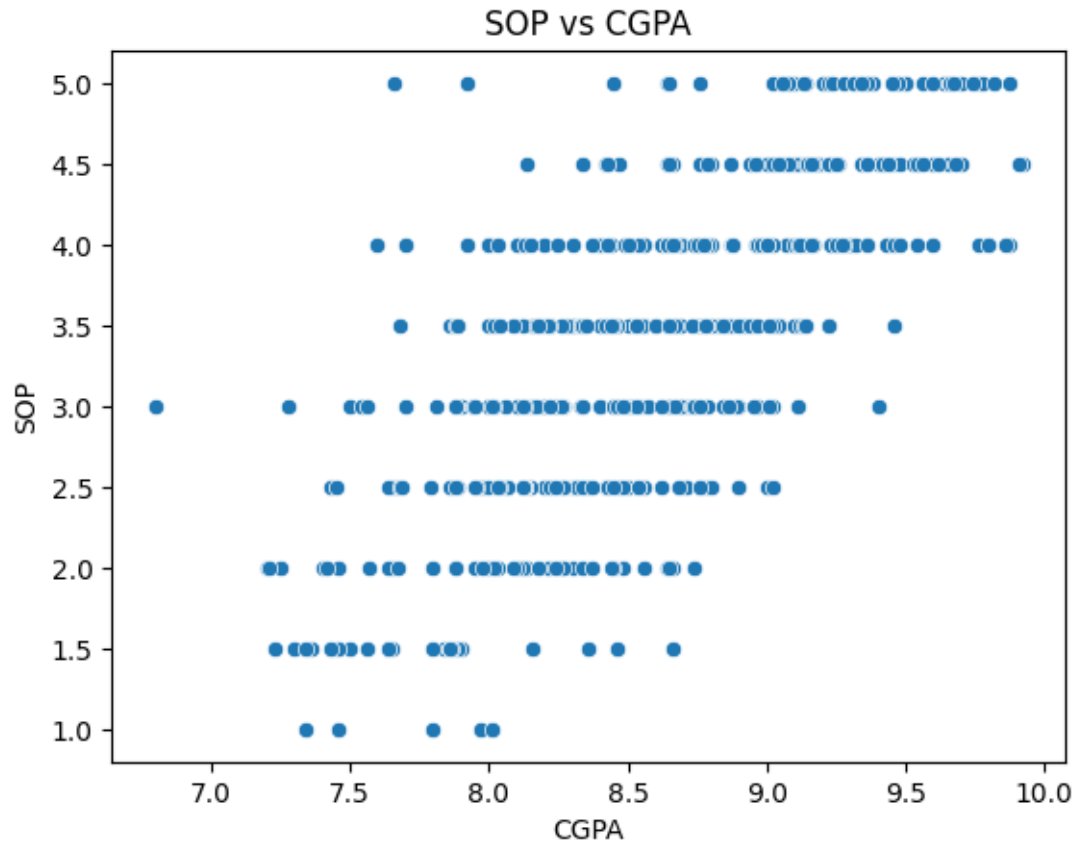
LORs are not that related with CGPA so it is clear that a persons LOR is not dependent on that persons academic excellence. Having research experience is usually related with a good LOR which might be justified by the fact that supervisors have personal interaction with the students performing research which usually results in good LORs

```
[28]: fig = sns.scatterplot(x="GRE Score", y="LOR ", data=df, hue="Research")
plt.title("GRE Score vs CGPA")
plt.show()
```



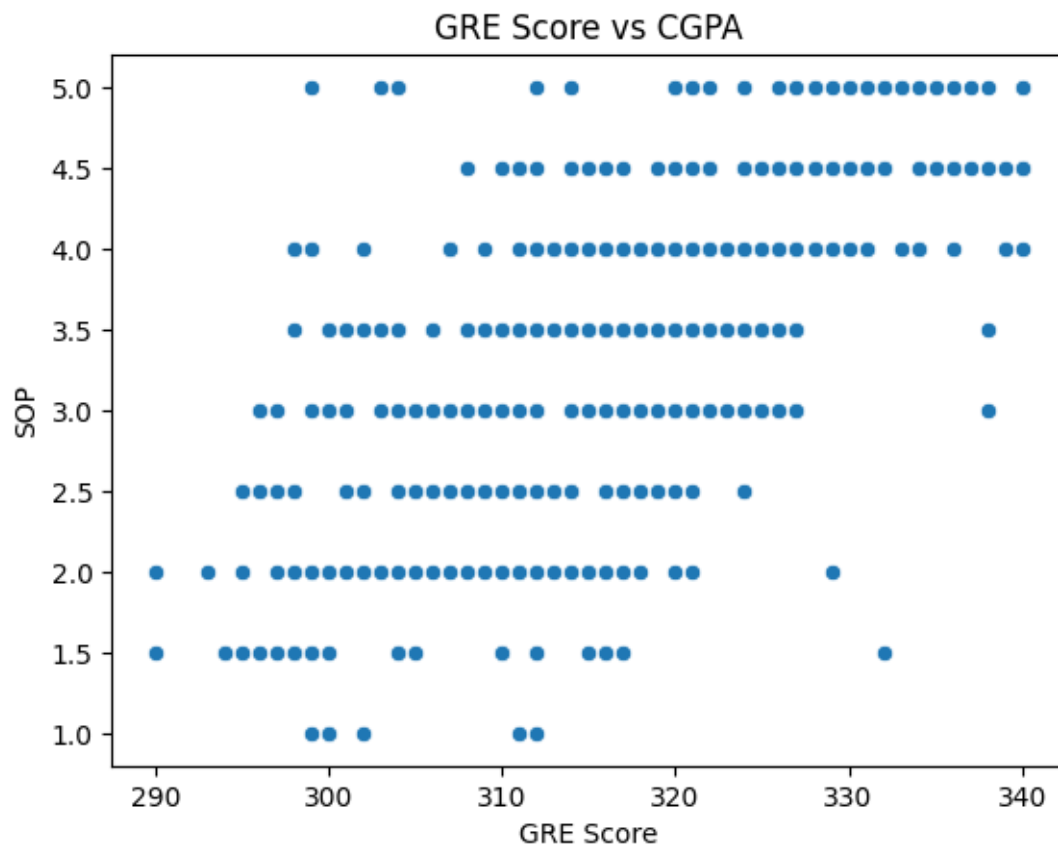
GRE scores and LORs are also not that related. People with different kinds of LORs have all kinds of GRE scores

```
[29]: fig = sns.scatterplot(x="CGPA", y="SOP", data=df)
plt.title("SOP vs CGPA")
plt.show()
```

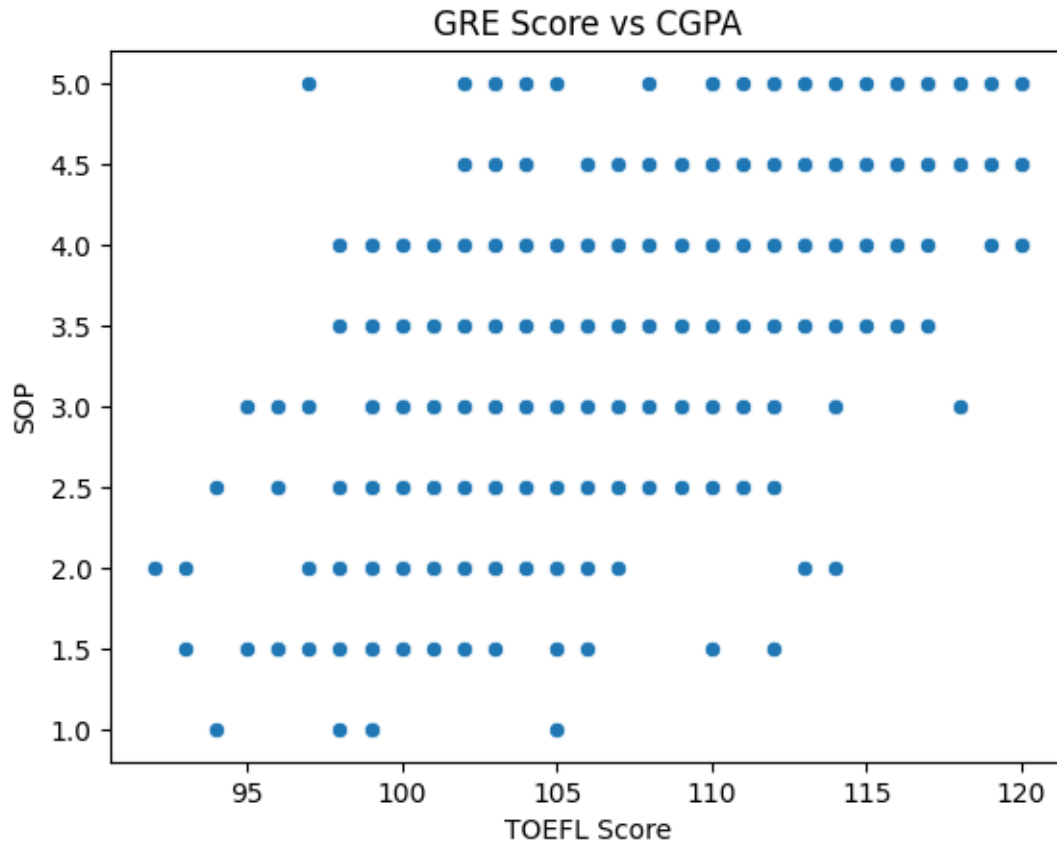


CGPA and SOP are not that related because Statement of Purpose is related to academic performance, but since people with good CGPA tend to be more hard working so they have good things to say in their SOP which might explain the slight move towards higher CGPA as along with good SOPs

```
[30]: fig = sns.scatterplot(x="GRE Score", y="SOP", data=df)
plt.title("GRE Score vs CGPA")
plt.show()
```



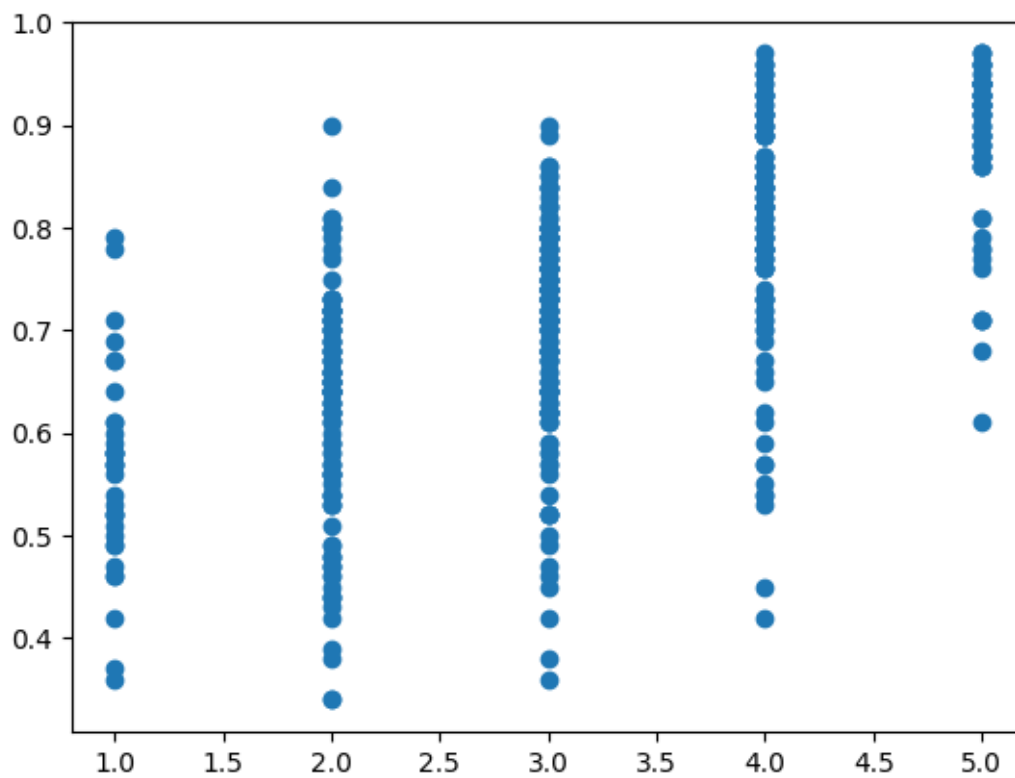
```
[31]: fig = sns.scatterplot(x="TOEFL Score", y="SOP", data=df)
plt.title("GRE Score vs CGPA")
plt.show()
```



- Applicants with different kinds of SOP have different kinds of TOEFL Score. So the quality of SOP is not always related to the applicants English skills.
- The Plot between CGPA and Chance of Admit shows that students having Higher CGPA have a high chance of getting admission to dollowing university.

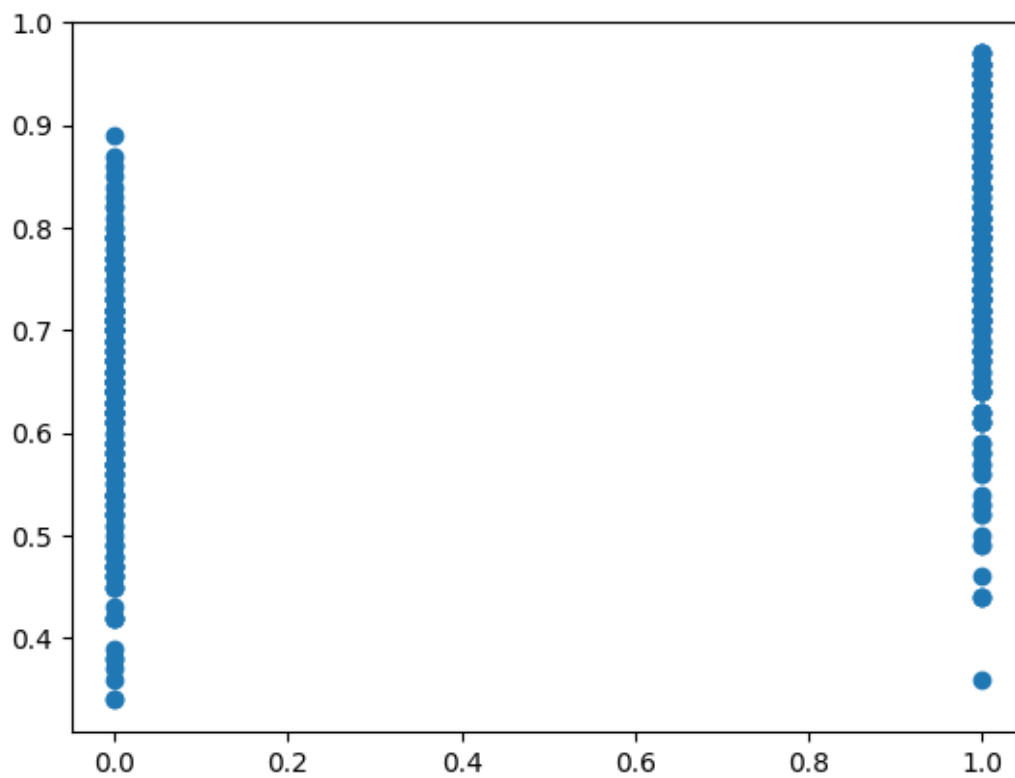
```
[32]: plt.scatter(x = df["University Rating"], y = df["Chance of Admit "])
```

```
[32]: <matplotlib.collections.PathCollection at 0x2b90e530550>
```



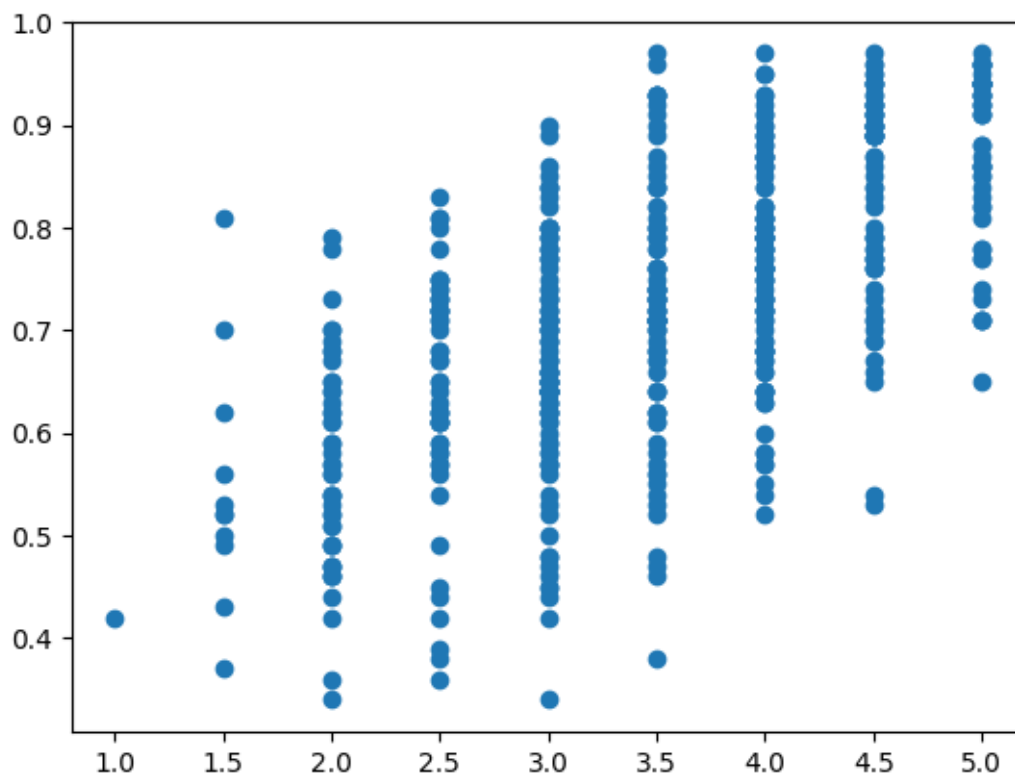
```
[33]: plt.scatter(x = df["Research"], y = df["Chance of Admit "])
```

```
[33]: <matplotlib.collections.PathCollection at 0x2b90e5a82d0>
```



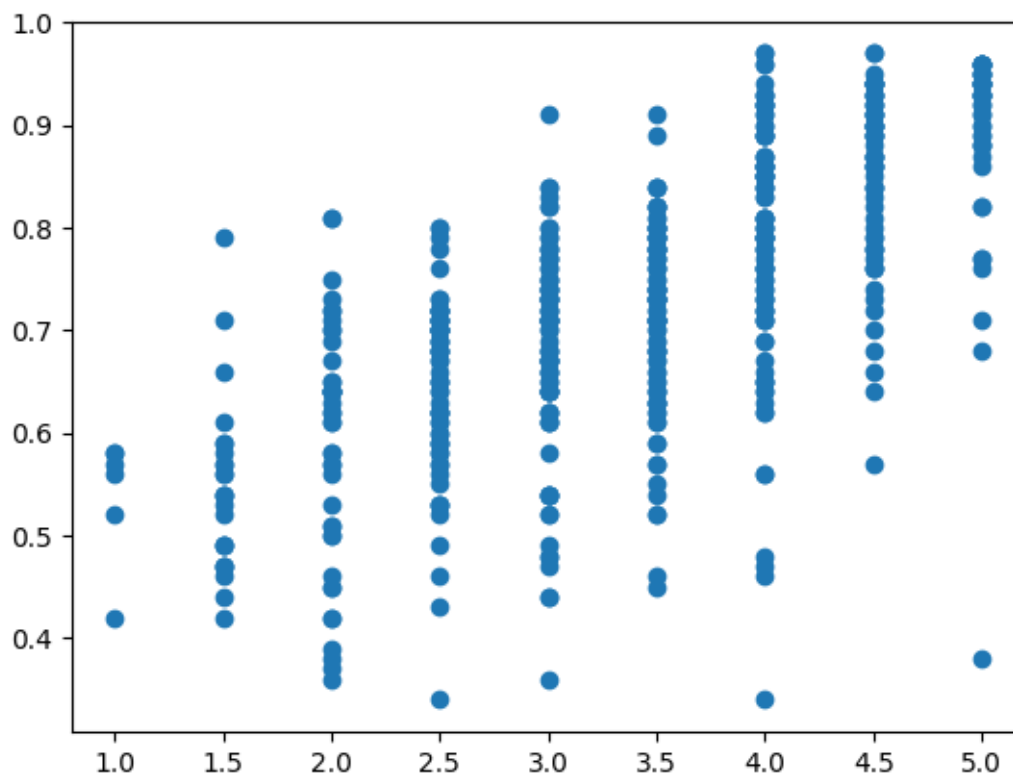
```
[34]: plt.scatter(x = df["LOR "], y = df["Chance of Admit "])
```

```
[34]: <matplotlib.collections.PathCollection at 0x2b90e60a850>
```

```
[35]: plt.scatter(x = df["SOP"], y = df["Chance of Admit "])
```

```
[35]: <matplotlib.collections.PathCollection at 0x2b90f6aa490>
```



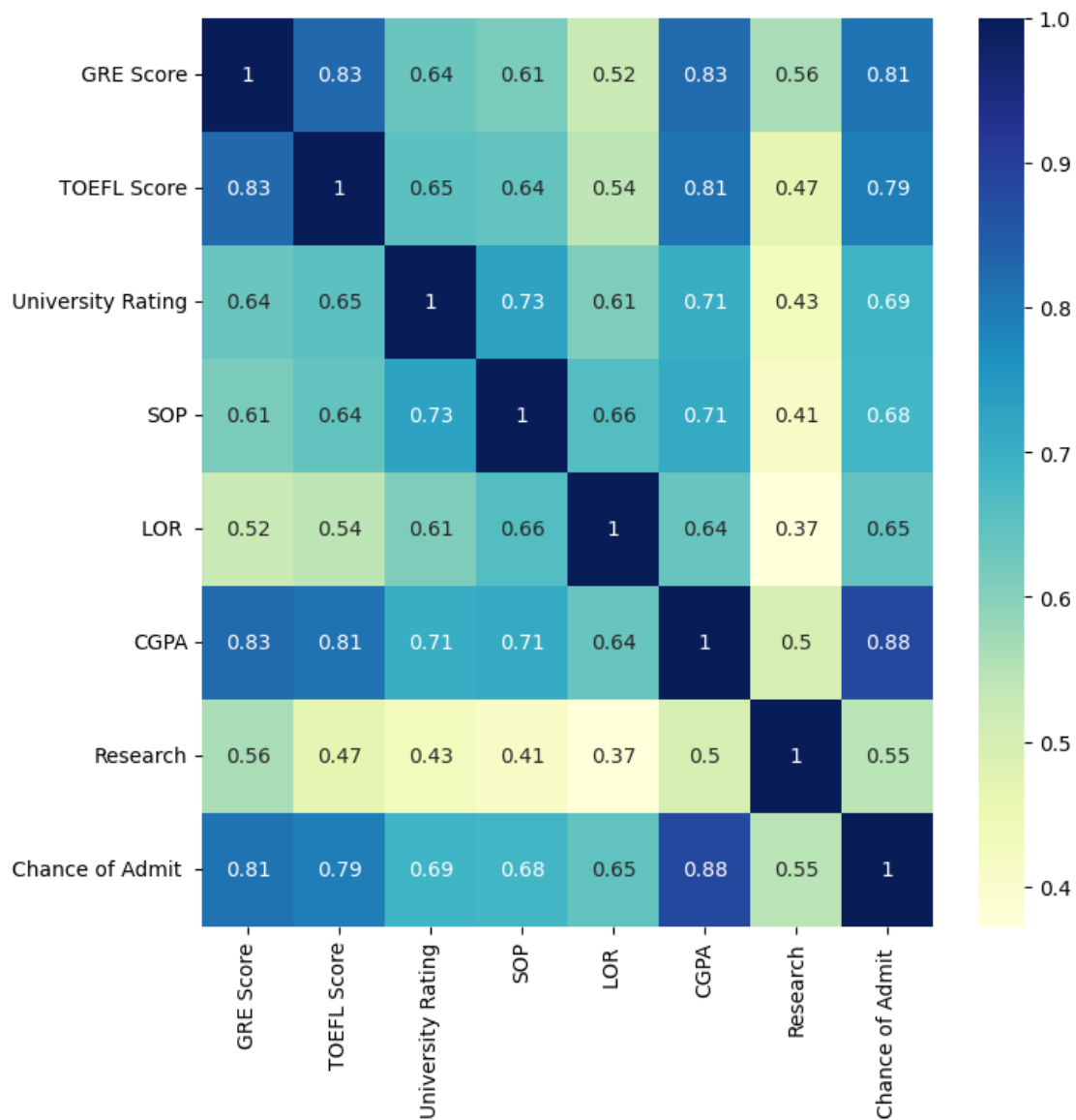
```
[36]: df.corr()
```

```
[36]:
```

	GRE Score	TOEFL Score	University Rating	SOP \
GRE Score	1.000000	0.827200	0.635376	0.613498
TOEFL Score	0.827200	1.000000	0.649799	0.644410
University Rating	0.635376	0.649799	1.000000	0.728024
SOP	0.613498	0.644410	0.728024	1.000000
LOR	0.524679	0.541563	0.608651	0.663707
CGPA	0.825878	0.810574	0.705254	0.712154
Research	0.563398	0.467012	0.427047	0.408116
Chance of Admit	0.810351	0.792228	0.690132	0.684137

	LOR	CGPA	Research	Chance of Admit
GRE Score	0.524679	0.825878	0.563398	0.810351
TOEFL Score	0.541563	0.810574	0.467012	0.792228
University Rating	0.608651	0.705254	0.427047	0.690132
SOP	0.663707	0.712154	0.408116	0.684137
LOR	1.000000	0.637469	0.372526	0.645365
CGPA	0.637469	1.000000	0.501311	0.882413
Research	0.372526	0.501311	1.000000	0.545871
Chance of Admit	0.645365	0.882413	0.545871	1.000000

```
[37]: fig, ax = plt.subplots(figsize=(8,8))
      dataplot = sns.heatmap(df.corr(), cmap="YlGnBu", annot=True)
```



1. There is a high correlation between GRE Score, TOEFL Score and Chance of Admit
2. There is a some correlation between University Rating, SOP, LOR and Chance of Admit
3. There is a very high correlation between CGPA and Chance of Admit
4. Research column has least effect in comparision to all others on Chances of Admission

0.6 Data Preprocessing & Model Building

```
[38]: import statsmodels.api as sm
```

```
[39]: df
```

```
[39]:
```

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	\
0	337	118	4	4.5	4.5	9.65	1	
1	324	107	4	4.0	4.5	8.87	1	
2	316	104	3	3.0	3.5	8.00	1	
3	322	110	3	3.5	2.5	8.67	1	
4	314	103	2	2.0	3.0	8.21	0	
..	...	...	...	...	...	...	...	
495	332	108	5	4.5	4.0	9.02	1	
496	337	117	5	5.0	5.0	9.87	1	
497	330	120	5	4.5	5.0	9.56	1	
498	312	103	4	4.0	5.0	8.43	0	
499	327	113	4	4.5	4.5	9.04	0	

	Chance of Admit
0	0.92
1	0.76
2	0.72
3	0.80
4	0.65
..	...
495	0.87
496	0.96
497	0.93
498	0.73
499	0.84

[500 rows x 8 columns]

Dividing the given data in 2 parts -

X - Features including Columns (GRE_score, TOFEL score etc.)

Y - Target as (Chance of Admit) :

```
[40]: y=df[['Chance of Admit ']]  
X=df.drop('Chance of Admit ', axis=1)
```

Lets split the dataset with training and testing set and prepare the inputs and outputs: Dividing the data into Training (70%) and Testing(30%) data.

```
[41]: from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,  
↪shuffle=True)
```

```
[42]: X_train
```

```
[42]:      GRE Score  TOEFL Score  University Rating  SOP  LOR  CGPA  Research
186      317      107           3  3.5  3.0  8.68           1
31      327      103           3  4.0  4.0  8.30           1
13      307      109           3  4.0  3.0  8.00           1
138      326      116           2  4.5  3.0  9.08           1
103      317      104           2  4.5  4.0  8.47           0
..      ...      ...           ...  ...  ...  ...           ...
385      335      117           5  5.0  5.0  9.82           1
144      325      112           2  3.0  3.5  8.96           1
328      324      112           4  4.0  3.5  8.77           1
253      335      115           4  4.5  4.5  9.68           1
139      318      109           1  3.5  3.5  9.12           0
```

[400 rows x 7 columns]

```
[45]: y_train
```

```
[45]:      Chance of Admit
186      0.84
31      0.74
13      0.62
138      0.80
103      0.57
..      ...
385      0.96
144      0.80
328      0.80
253      0.93
139      0.78
```

[400 rows x 1 columns]

```
[46]: #Standardization
from sklearn.preprocessing import StandardScaler
X_train_columns=X_train.columns
std=StandardScaler()
X_train_std=std.fit_transform(X_train)
```

```
[47]: X_train_std
```

```
[47]: array([[ -0.01694209, -0.08842821, -0.14470371, ..., -0.5644136 ,
          0.11769111,  0.86855395],
        [ 0.87474683, -0.74957372, -0.14470371, ...,  0.51580381,
        -0.51176172,  0.86855395],
        [-0.90863101,  0.24214454, -0.14470371, ..., -0.5644136 ,
```

```

-1.00869816, 0.86855395],
...,
[ 0.60724015, 0.73800368, 0.74578068, ..., -0.02430489,
 0.26677205, 0.86855395],
[ 1.58809796, 1.23386281, 0.74578068, ..., 1.05591252,
 1.77414593, 0.86855395],
[ 0.0722268 , 0.24214454, -1.9256725 , ..., -0.02430489,
 0.84653123, -1.15133896]], shape=(400, 7))

```

```
[48]: X_train=pd.DataFrame(X_train_std, columns=X_train_columns)
```

```
[49]: X_train
```

```
[49]:
```

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	\
0	-0.016942	-0.088428	-0.144704	0.083255	-0.564414	0.117691	
1	0.874747	-0.749574	-0.144704	0.587831	0.515804	-0.511762	
2	-0.908631	0.242145	-0.144704	0.587831	-0.564414	-1.008698	
3	0.785578	1.399149	-1.035188	1.092407	-0.564414	0.780273	
4	-0.016942	-0.584287	-1.035188	1.092407	0.515804	-0.230164	
..	...	...	...	...	...	...	
395	1.588098	1.564436	1.636265	1.596983	1.596021	2.006050	
396	0.696409	0.738004	-1.035188	-0.421321	-0.024305	0.581498	
397	0.607240	0.738004	0.745781	0.587831	-0.024305	0.266772	
398	1.588098	1.233863	0.745781	1.092407	1.055913	1.774146	
399	0.072227	0.242145	-1.925673	0.083255	-0.024305	0.846531	

```

Research
0    0.868554
1    0.868554
2    0.868554
3    0.868554
4   -1.151339
..
395  0.868554
396  0.868554
397  0.868554
398  0.868554
399 -1.151339

```

```
[400 rows x 7 columns]
```

Let's train our model using StatsModel library Lets use a bunch of different algorithms to see which model performs better.

```
[50]: from sklearn.metrics import accuracy_score
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Lasso,Ridge,LinearRegression
from sklearn.metrics import mean_squared_error
```

```
[53]: models = [
        ['Linear Regression :', LinearRegression()],

        ['Lasso Regression :', Lasso(alpha=0.1)], #try with different alpha
        ↪values
        ['Ridge Regression :', Ridge(alpha=1.0)] #try with different alpha
        ↪values
    ]

print("Results without removing features with multicollinearity ...")

for name,model in models:
    model.fit(X_train, y_train.values)
    predictions = model.predict(std.transform(X_test))
    print(name, (np.sqrt(mean_squared_error(y_test, predictions))))

import warnings
warnings.filterwarnings("ignore", message="X does not have valid feature names")
```

Results without removing features with multicollinearity ...

Linear Regression : 0.06513627439591406

Lasso Regression : 0.1328006881029149

Ridge Regression : 0.06513748754223751

Terms involved in the LR & Fitting the Model:

- Dep. Variable: This column displays the name of the dependent variable being predicted in the regression. (y in our case)
- Model: It provides a concise representation of the model type and method used, such as “OLS” (Ordinary Least Squares).
- R-squared: Represents the coefficient of determination (R-squared) value.
- Adj. R-squared: This is the adjusted R-squared value, which accounts for the number of predictors in the model and adjusts the R-squared accordingly.
- Adjusted. R-squared reflects the fit of the model. R-squared values range from 0 to 1, where a higher value generally indicates a better fit, assuming certain conditions are met.
- const coefficient is your Y-intercept. It means that if both the Interest_Rate and Unemployment_Rate coefficients are zero, then the expected output (i.e., the Y) would be equal to the const coefficient.
- std err reflects the level of accuracy of the coefficients. The lower it is, the higher is the level of accuracy.
- $P > |t|$ is your p-value. A p-value of less than 0.05 is considered to be statistically significant.
- Confidence Interval represents the range in which our coefficients are likely to fall (with a likelihood of 95%)

```
[54]: # Statmodels implementation of Linear regression
import statsmodels.api as sm
Xm_train = sm.add_constant(X_train)
model = sm.OLS(y_train.values, Xm_train).fit()
print(model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  y      R-squared:                  0.823
Model:                          OLS    Adj. R-squared:             0.820
Method:                        Least Squares    F-statistic:                260.6
Date:                          Tue, 30 Sep 2025    Prob (F-statistic):        3.47e-143
Time:                           08:48:35    Log-Likelihood:            569.94
No. Observations:                400    AIC:                       -1124.
Df Residuals:                    392    BIC:                       -1092.
Df Model:                        7
Covariance Type:                nonrobust
=====
=====
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
-----
const                0.7309      0.003    248.612    0.000      0.725
0.737
GRE Score            0.0209      0.007      3.189    0.002      0.008
0.034
TOEFL Score          0.0147      0.006      2.489    0.013      0.003
0.026
University Rating    0.0086      0.005      1.784    0.075     -0.001
0.018
SOP                  0.0008      0.005      0.154    0.878     -0.009
0.011
LOR                  0.0122      0.004      2.830    0.005      0.004
0.021
CGPA                 0.0722      0.006     11.374    0.000      0.060
0.085
Research             0.0119      0.004      3.228    0.001      0.005
0.019
=====
=====
Omnibus:                104.576    Durbin-Watson:              2.164
Prob(Omnibus):           0.000    Jarque-Bera (JB):           272.708
Skew:                    -1.260    Prob(JB):                    6.06e-60
Kurtosis:                 6.165    Cond. No.                     5.90
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly

specified.

```
[55]: Xm_train
```

```
[55]:      const  GRE Score  TOEFL Score  University Rating      SOP      LOR  \
0      1.0   -0.016942   -0.088428      -0.144704  0.083255 -0.564414
1      1.0    0.874747   -0.749574      -0.144704  0.587831  0.515804
2      1.0   -0.908631    0.242145      -0.144704  0.587831 -0.564414
3      1.0    0.785578    1.399149      -1.035188  1.092407 -0.564414
4      1.0   -0.016942   -0.584287      -1.035188  1.092407  0.515804
..      ...      ...      ...      ...      ...      ...
395     1.0    1.588098    1.564436      1.636265  1.596983  1.596021
396     1.0    0.696409    0.738004     -1.035188 -0.421321 -0.024305
397     1.0    0.607240    0.738004     0.745781  0.587831 -0.024305
398     1.0    1.588098    1.233863     0.745781  1.092407  1.055913
399     1.0    0.072227    0.242145     -1.925673  0.083255 -0.024305
```

```
      CGPA  Research
0    0.117691  0.868554
1   -0.511762  0.868554
2   -1.008698  0.868554
3    0.780273  0.868554
4   -0.230164 -1.151339
..      ...      ...
395  2.006050  0.868554
396  0.581498  0.868554
397  0.266772  0.868554
398  1.774146  0.868554
399  0.846531 -1.151339
```

[400 rows x 8 columns]

```
[56]: Xm_train_new=Xm_train.drop(columns='SOP')
model1 = sm.OLS(y_train.values, Xm_train_new).fit()
print(model1.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          y      R-squared:          0.823
Model:                  OLS    Adj. R-squared:      0.820
Method:                 Least Squares  F-statistic:    304.8
Date:                   Tue, 30 Sep 2025  Prob (F-statistic): 1.93e-144
Time:                   08:48:53  Log-Likelihood:   569.92
No. Observations:      400      AIC:             -1126.
Df Residuals:          393      BIC:             -1098.
Df Model:               6
Covariance Type:       nonrobust
=====
```

```

=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
const          0.7309      0.003    248.922    0.000      0.725
0.737
GRE Score      0.0209      0.007      3.193    0.002      0.008
0.034
TOEFL Score    0.0148      0.006      2.508    0.013      0.003
0.026
University Rating 0.0089      0.004      2.015    0.045      0.000
0.018
LOR            0.0124      0.004      3.031    0.003      0.004
0.020
CGPA           0.0724      0.006     11.543    0.000      0.060
0.085
Research       0.0120      0.004      3.249    0.001      0.005
0.019
=====
Omnibus:                104.249    Durbin-Watson:                2.165
Prob(Omnibus):           0.000    Jarque-Bera (JB):             271.532
Skew:                   -1.256    Prob(JB):                     1.09e-59
Kurtosis:                6.159    Cond. No.                      5.46
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The advantage of using StatsModel is that it is providing us with the R squared and adjusted R squared values directly:

1. As we see in Model1, where we drop “SOP” feature column we see clear difference between R squared and Adj. R squared values, among the models model (R-squared:0.0.836; Adj. R-squared:0.833)

Model1 (R-squared:0.836; Adj. R-squared:0.833)

0.7 Testing the assumptions of the linear regression model

Linear regression, a fundamental statistical method for modeling the relationship between a dependent variable and one or more independent variables, relies on several key assumptions.

We can prove that Linear regression is a very good model if all the statistical assumptions holds true.

Assumptions of Linear Regression

1. **Linearity:** The relationship between the independent variables and the dependent variable is linear. This means the change in the dependent variable due to a one-unit change in an independent variable is constant.
2. **Independence:** Observations are independent of each other. This is crucial for the reliability of the statistical inferences made from the model.
3. **Homoscedasticity:** The variance of residual errors is constant across all levels of the independent variables. If the variance change (heteroscedasticity), it can lead to inefficient estimates.
4. **Normal Distribution of Errors:** The residuals (errors) of the model are normally distributed. This assumption facilitates the creation of confidence intervals and hypothesis tests.
5. **No or Little Multicollinearity:** Independent variables should not be too highly correlated with each other. High correlation can make it difficult to isolate the effect of each independent variable on the dependent variable. Let's check them one by one.

It is important to Scale the data, so using STANDARD SCALER for it.

0.8 Multicollinearity Check and VIF (Variance Inflation Factor)

- VIF score of an independent variable represents how well the variable is explained by other independent variables.
- When one or more input variables exhibit strong correlation and One variable is enough to be part of the model.
- So, the closer the R^2 value to 1, the higher the value of VIF and the higher the multicollinearity with the particular independent variable.

0.9 Normal Distribution of Errors:

```
[59]: y_train_Array = np.array(y_train)
```

```
[60]: X = df[df.columns.drop('Chance of Admit ')]
y = df["Chance of Admit "]

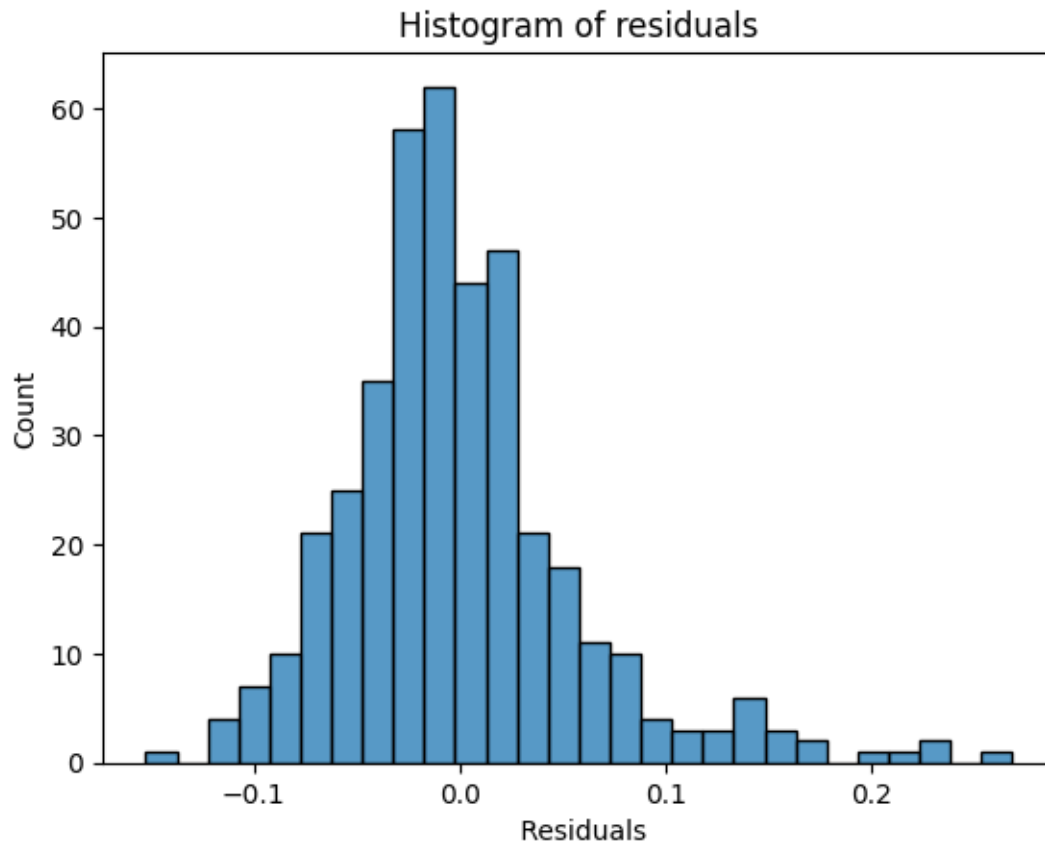
X_train, X_test, y_train_Array, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=2)

scaler = StandardScaler()
X_tr_scaled = scaler.fit_transform(X_train)
X_sm = sm.add_constant(X_tr_scaled)
sm_model = sm.OLS(y_train_Array, X_sm).fit()
```

```
[61]: Y_hat = sm_model.predict(X_sm)
errors = Y_hat - y_train_Array
```

```
[62]: sns.histplot(errors)
plt.xlabel(" Residuals")
plt.title("Histogram of residuals")
```

```
[62]: Text(0.5, 1.0, 'Histogram of residuals')
```



```
[64]: from scipy import stats
res = stats.shapiro(errors)
res.statistic
```

```
[64]: np.float64(0.9314318654549102)
```

```
[65]: res
```

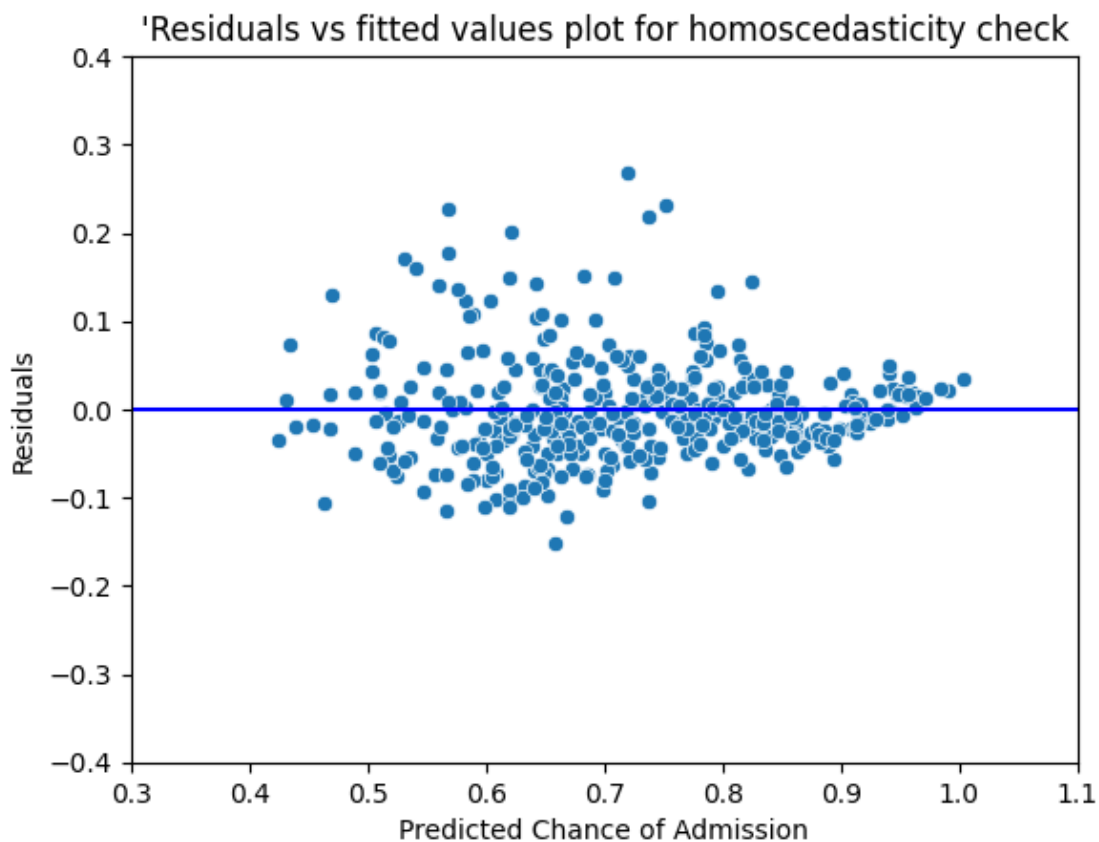
```
[65]: ShapiroResult(statistic=np.float64(0.9314318654549102),
pvalue=np.float64(1.3679305929637856e-12))
```

- Closer the value to 1, more is the normality.
- In this case, a value of 0.93 denotes a high level of normality for the error distribuiton
- If the distribution of errors is gaussian distribution then the assumption of error being normally distributed is being sastified (sanity check)
- Heteroskadestacity shouldn't exist

```
[66]: Y_hat = sm_model.predict(X_sm)
errors = Y_hat - y_train_Array
```

```
[67]: sns.scatterplot(x=Y_hat,y=errors)
plt.xlabel("Predicted Chance of Admission")
plt.ylabel("Residuals")
plt.ylim(-0.4,0.4)
plt.xlim(0.3,1.1)
p = sns.lineplot(x=[0,26], y=[0,0], color='blue')
plt.title("'Residuals vs fitted values plot for homoscedasticity check")
```

```
[67]: Text(0.5, 1.0, "'Residuals vs fitted values plot for homoscedasticity check")
```



```
[68]: # Performing the Goldfeld-Quandt test to check for Homoscedasticity -
from statsmodels.compat import lzip
import statsmodels.stats.api as sms

name = ['F statistic', 'p-value']
test = sms.het_goldfeldquandt(y_train_Array,X_sm )
lzip(name, test)
```

```
[68]: [('F statistic', np.float64(1.0772994279987234)),
      ('p-value', np.float64(0.30323276479816175))]
```

From the goldfeld-quandt test:

- F Statistic comes out to be 1.0772 => Implying minimal difference in variance between groups
- p-value of 0.3032 indicates that this difference is statistically significant at conventional levels of significance (e.g., 0.05).

Therefore, we accept the null hypothesis of homoscedasticity, and conclude that there is no strong evidence of heteroscedasticity in the data.

0.10 Applying Lasso & Ridge regularization

```
[69]: from sklearn.linear_model import Lasso, Ridge
      from sklearn.linear_model import LinearRegression
      from sklearn.preprocessing import PolynomialFeatures
```

```
[70]: # Split the data into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      random_state=42)
```

```
[71]: # Transform the features into polynomial features
      degree = 5
      poly = PolynomialFeatures(degree=degree)
      X_train_poly = poly.fit_transform(X_train)
      X_test_poly = poly.transform(X_test)
```

```
[72]: # Standardize the polynomial features
      scaler = StandardScaler()
      X_train_poly_scaled = scaler.fit_transform(X_train_poly)
      X_test_poly_scaled = scaler.transform(X_test_poly)
```

```
[73]: model = LinearRegression()
      model.fit(X_train_poly_scaled , y_train)

      output = model.predict(X_test_poly_scaled)
```

Taking MSE as metric instead of Adj.R-sq

```
[74]: from sklearn.metrics import mean_squared_error

      print('MSE for test:', mean_squared_error(y_test, output))

      output = model.predict(X_train_poly_scaled)
      print('MSE for train:', mean_squared_error(y_train, output))
```

MSE for test: 6.752280976233891

MSE for train: 4.425356111970129e-22

Clearly degree = 5 overfits in the model, increasing the MSE.

```
[75]: lasso_model = Lasso(alpha=0.01)  # Alpha is the regularization strength
      ridge_model = Ridge(alpha=1.0)   # Alpha is the regularization strength
```

```
[76]: # Fit the models to the training data
      lasso_model.fit(X_train_poly_scaled, y_train)
      ridge_model.fit(X_train_poly_scaled, y_train)
```

```
[76]: Ridge()
```

```
[77]: Ridge()
```

```
[77]: Ridge()
```

```
[78]: lasso_predictions = lasso_model.predict(X_train_poly_scaled)
      ridge_predictions = ridge_model.predict(X_train_poly_scaled)

      print('test MSE for L1:', mean_squared_error(y_train, lasso_predictions))
      print('test MSE for L2:', mean_squared_error(y_train, ridge_predictions))
```

```
test MSE for L1: 0.0036923994575747588
test MSE for L2: 0.0030771485977503688
```

```
[79]: lasso_predictions = lasso_model.predict(X_test_poly_scaled)
      ridge_predictions = ridge_model.predict(X_test_poly_scaled)

      print('test MSE for L1:', mean_squared_error(y_test, lasso_predictions))
      print('test MSE for L2:', mean_squared_error(y_test, ridge_predictions))
```

```
test MSE for L1: 0.0037294101004085502
test MSE for L2: 0.0037841315629941656
```

Clearly the MSE values have been reduced drastically by applying L1(Lasso) and L2(Ridge) regularization methods.

0.11 Conclusion

We can say that provided data can be modeled using Linear Regression model.

The data provided has no unwanted features so we can use all of those for the prediction of their impact on the Chances of getting admission.

1. Students having high GRE Score and TOEFL Score have high chances of getting admission in Universities with high rating.
2. Students securing high CGPA have very high chance of admission.
3. Research is not a very important feature to get admission.

0.12 Recommendations

1. Students looking for Admission in the Ivy league colleges should aim for high TOEFL, GRE and CGPA.
2. Universities can be identified by looking at their ratings.
3. Jamboree can use the above analysis to mention on their website the important factors for admission in Ivy league colleges.
4. Jamboree can help the students improve TOEFL, GRE and CGPA score of the students by introducing practice papers, mock test, etc.

[]: